COSC-524 Natural Language Processing Project 2: Fine-Tuning BERT

Sujit Tripathy, Cameron Rader, Befikir Bogale, William Parham
University of Tennessee, Knoxville

***Abstract*—This project explores the performance of the bert-base-uncased model, a variant of pretrained Bidirectional Encoder Representations from Transformers (BERT), and its fine-tuned versions for question-answering (QA) tasks using A Study in Scarlet by Sir Arthur Conan Doyle as the text corpus. We implemented two approaches: (1) Retrieval-Augmented Generation (RAG) to overcome BERT's 512-token limit by dynamically retrieving relevant text chunks for enhanced context generation, and (2) fine-tuning BERT with manually and automatically created QA datasets. Five chunking strategies (sentence, paragraph, scene, phrase, and semantic chunking) were tested for the RAG approach, with embeddings generated using sbert for efficiency. Fine-tuning involved QA datasets created via manual annotation and automated methods, to assess the trade-off between data quality and quantity. Evaluation metrics, including BERTScore, Exact Match, and F1, revealed that fine-tuned BERT outperformed the pretrained model. However, the RAG-based approach showed limited improvement due to BERT's token constraints and challenges in aligning retrieved contexts with answers.**

Keywords: Fine-Tuning, Retrieval Augmented Generation, BERT, Embeddings, Large Language Model

I. Introduction

This project compares the performance of a pretrained 'Bidirectional Encoder Representations from Transformers' (BERT) to fine-tuned versions for question answering (QA) tasks. The variant of BERT, we have used for this project, is the 'bert-base-uncased' model. Additionally, we have experimented with multiple custom implementations of Retrieval Augmented Generation (RAG) to see if enhanced context helps with the limitation of bert-base-uncased's small token size and will generate improved responses compared to the pretrained version. For this study, we have used the text *A Study in Scarlet* by Sir Arthur Conan Doyle [3].

Before we delve deeper into the methodology, we present the BERT variant 'bert-base-uncased' [1]. This is a general purpose large language model (LLM) developed by Google primarily pretrained with two objectives: masked language modeling (MLM) and next sentence prediction (NSP). The model has 110 million parameters, can process the 'English' language and has a maximum token size of 512 tokens. Uncased means that the model does not differentiate between uppercase and lowercase characters, for example, 'English' is equivalent to 'english'. Additionally, it was pretrained on the *BookCorpus* dataset, which consists of 11,038 unpublished books and English Wikipedia. Reference [1] recommends fine-tuning the model for various downstream tasks, including QA. **Figure 1** shows the architecture of BERT presented in [4].

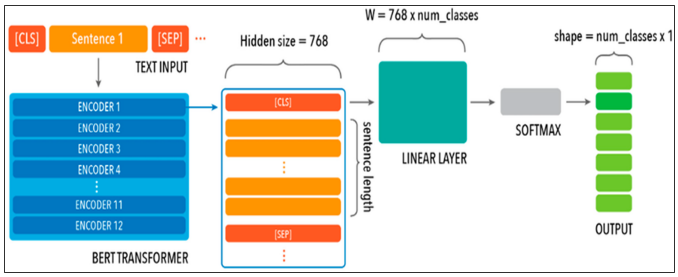

Fig. 1. BERT *architecture*

II. Methodology

We have implemented two approaches as mentioned before: a RAG-based approach for enhanced context generation, and fine-tuning bert-base-uncased using a QA dataset and comparing its performance with the pretrained version. Both these approaches are discussed next.

A. Retrieval Augmented Generation (RAG)-Based Approach

We have utilized RAG to provide enhanced contexts to the bert-base-uncased model for QA tasks [6]. This approach aims to address the 512-token limit of the BERT model by identifying and retrieving the most relevant contexts from the text to include within its context window given the question. It achieves this by comparing the text embedding of the query with the embeddings of the chunks created from the text, *A Study in Scarlet*. The comparison relies on cosine similarity, to allow RAG to identify the top 'k' chunks with the most similar embeddings from the document and use them to generate contexts. To optimize the use of the 512-token limit, we opted for a maximum chunk size of 256 tokens across different chunking strategies. **Figure 2** shows the general strategy employed by RAG [6]. Subsequently, the query along with the context are then passed on to the BERT model for generating responses.

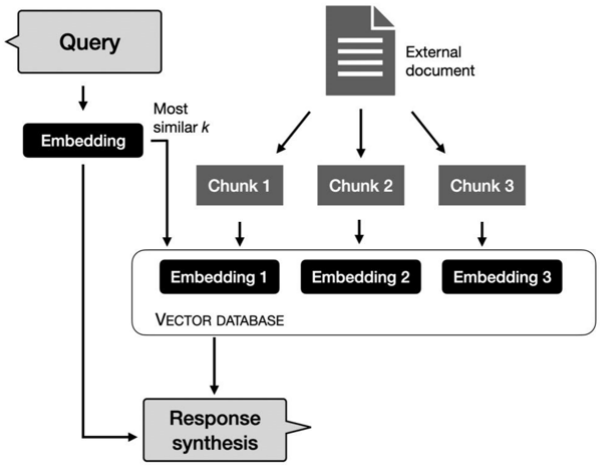

Fig. 2. RAG *architecture* [8]

To generate text embeddings for the RAG algorithm, we tested several models, including 'bert-base-uncased', 'sbert', and 'e5' [5]-[9]. Among these, we selected 'sbert' for its speed and efficiency. While 'e5' has been shown to deliver better results, it took approximately thirty times longer to encode chunks compared to 'sbert' and produced results that were similar [5]-[9].

We also explored five different chunking strategies: sentence, paragraph, scene, phrase, and semantic chunking [7]. These were tested to determine whether certain strategies might be better suited to different types of questions.

1) Sentence Chunking : Sentence chunking involves dividing the text into chunks by tokenizing it into sentences [7]. Each chunk is limited to a maximum size of 256 tokens, and an embedding is generated for each chunk. If a tokenized sentence exceeds 256 tokens, the first 256 tokens form one chunk, while the remainder is appended to the next sentence to create a new chunk (as long as the combined token length is under 256). This method was chosen as a baseline for comparison with other chunking strategies due to its simplicity of implementation. **Figure 3** displays the chunk counts per sentence resulting from sentence chunking. It highlights that each sentence is referenced only once by any chunk cumulatively.

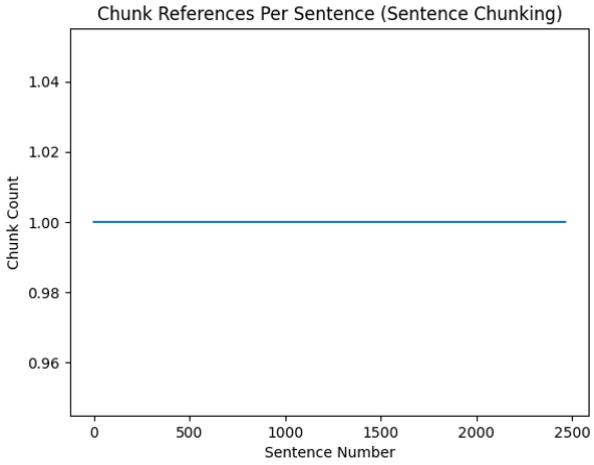

Fig. 3. Sentence *Chunking*

2) Paragraph Chunking : Paragraph chunking builds on sentence chunking by tokenizing the text at the paragraph level, with a maximum chunk size of 256 tokens. This method aims to preserve the semantic coherence of a paragraph while maintaining the spatial locality within the text [7]. Each chunk typically corresponds to a single paragraph, unless the paragraph exceeds 256 tokens, in which case it is split into multiple chunks. This approach is expected to perform better for longer, more detailed summary-type questions. **Figure 4** illustrates the paragraph chunking strategy. In this approach, a paragraph is usually referenced by a single chunk. However, as shown in the graph, some paragraphs are divided into eight different chunks, which occurs when a paragraph exceeds the 256-token limit.

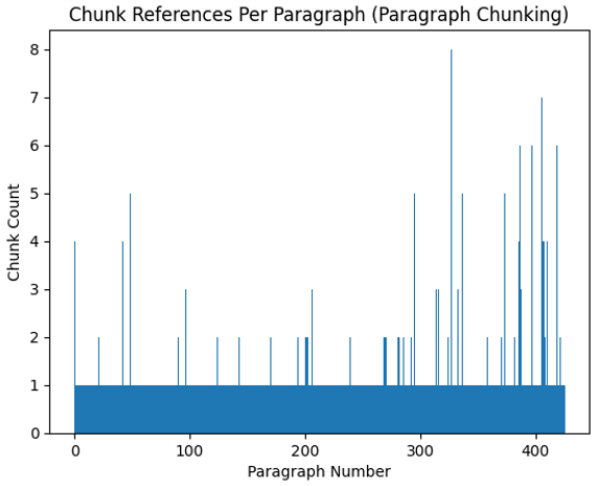

Fig. 4. Paragraph *Chunking*

3) Scene Chunking : Scene chunking applies some of the features we extracted from project 1 and aims to create more intelligent and semantically similar chunks. To do this, we created 256 token length chunks based on the 'scenes' we found in project 1. Here, a scene is dictated as 3 characters

appearing concurrently within a specific window length. This chunking method is expected to better handle character related questions as well as questions dealing with plot analysis such as when a murder happens.

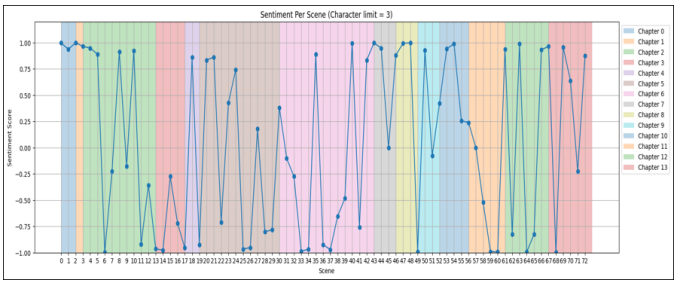

Fig. 5. Scene *Chunking*

Figure 5 illustrates the inspiration for our scene chunking approach. By calculating the sentiment scores of scenes we were able to deduce where major plot points occur. The figure shows this by demonstrating the high and low points within each scene, and how the scenes make up the length of the novel.

4) Phrase Chunking : Phrase chunking is a strategy developed to capture smaller phrases within the text. In this method, chunks are created in groups of four tokens, with a mapping to link each chunk back to its originating sentence. After generating embeddings for these chunks, the top 'k' most similar chunks to the query are identified. Instead of returning the raw phrases, each chunk is mapped back to its corresponding sentence, and these sentences are provided as context [7]. We observed this approach to excel at identifying short-answer questions, such as answering questions like "What was written on the wall?"

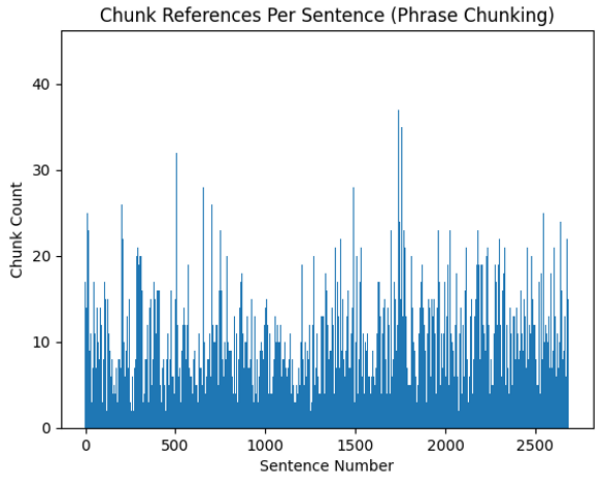

Fig. 6. Phrase *Chunking*

Figure 6 illustrates the phrase chunking strategy. Since each chunk consists of 4 tokens, a single sentence can be referenced multiple times. It is important to highlight that in phrase chunking, the chunks themselves do not form the context. Instead, the originating sentences containing the top-ranked chunks are used to construct the context.

5) Semantic Chunking : Semantic chunking is a strategy that generates chunks with a maximum length of 256 tokens by grouping semantically similar sentences. This is achieved by comparing the embeddings of sentences and combining them based on a similarity threshold. For further analysis, we explored: continuous and non-continuous semantic chunking. In continuous chunking, only adjacent sentences are chunked together. In non-continuous chunking, we use a k-means clustering algorithm to form a predefined number of clusters or chunks containing semantically similar sentences (1000 clusters for this work) [7].

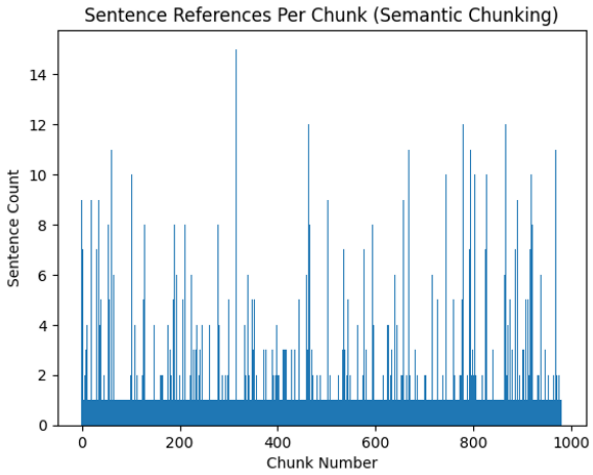

Fig. 7. Semantic *Chunking*

Figure 7 is notably different from the other figures. Please note the labels on the x and y axes. The plot represents the number of sentences contained within each chunk, generated using the continuous semantic chunking approach.

B. Fine-Tuning

We fine-tuned a pretrained BERT model to improve its understanding of the text, *A Study in Scarlet* by Sir Arthur Conan Doyle, aiming to enhance its ability to answer questions from the novel. This was done using datasets that were created from the text and comprised of context, question, and answer sets. To highlight the importance of both data quality and quantity for fine-tuning, we used a two fold approach for dataset generation: creating a high-quality QA dataset manually and generating an automated QA dataset using language models. The figure below illustrates the full process pipeline.

Figure 8 (*Fine-Tuning Pipeline*) depicts the fine-tuning process, displaying how the datasets are created and tokenized, the BERT model is fine-tuned, and the performance is evaluated against the pretrained BERT.

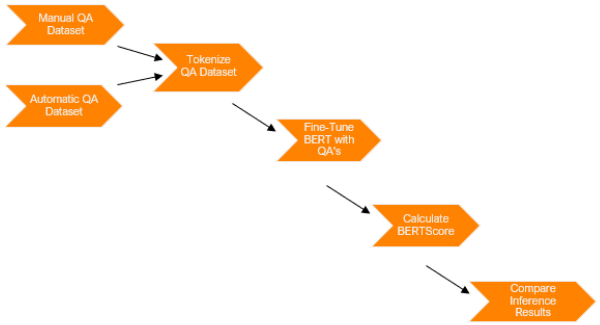

Fig. 8. Example *Dataset*

1) Creating QA Dataset Manually: We manually created a high-quality QA dataset consisting of approximately 300 context, question, and answer sets. Of these, around 250 data points were used for fine-tuning the BERT model, while the remaining were utilized for inference testing. The dataset included various types of questions, such as short-answer, summary-type, and factual questions, with varying context lengths. These were created to fine-tune the model using a variety of QA data types.

2) Automated Generation of QA Dataset: Manually creating the QA dataset was a painstaking and time-consuming process, though it resulted in high-quality data. To make the process more efficient, we explored other methods for generating QA datasets. One such approach involved scanning the text to identify key sentences, determined by the sum of the term frequency-inverse document frequency (TF-IDF) scores of the words they contained. These important sentences would then be used as contexts. Using the T5 model, questions would be generated based on these contexts, while named entities and nouns from the sentences were extracted to form a set of answers. For each combination of question, answer, and context, a QA set would be created. However, we opted not to use this method for automated QA generation due to the low quality of the generated QA datasets.

We adopted an alternative approach that utilized three different language models to independently generate contexts, questions, and answers for each data point. For context creation, Google's Pegasus-large model (maximum token length of 1024) was used to summarize large sections of text, specifically sentences from A Study in Scarlet, into concise pieces of context. The T5 model (maximum token length of 512) was then used to generate questions based on these summaries, and the RoBERTa-large-squad2 model (maximum token length of 384) was employed to produce answers for the corresponding question-context pairs. The models were executed iteratively, processing a specified number of sentences per iteration to create the elements of each QA data sample.

This method generated a dataset of over 700 samples, although the quality was lower compared to the manually created dataset. We spot-checked the automatically generated QA dataset but did not use any evaluation metrics to assess its quality, as this was outside the scope of the project. Ultimately, this allowed us to explore what has a greater impact on fine-tuning: a larger dataset with lower-quality questions or a smaller, manually created dataset with higher-quality questions.

Figure 9 (*Example Dataset*) shows some examples from the manually created QA dataset (in .json format file), that includes context, question, answer, and the character-based start and end positions of the answer within the context.


```
[
    {
        "context": "Jefferson Hope, the cab driver, was revealed to be the murderer.
        "question": "Who was revealed to be the murderer?",
        "answer": "Jefferson Hope",
        "start_position": 0,
        "end_position": 14
    },
    {
        "context": "Hope was motivated by a desire for revenge against Enoch Drebber
        "question": "What motivated Jefferson Hope's actions?",
        "answer": "revenge against Enoch Drebber",
        "start_position": 35,
        "end_position": 64
    },
    {
        "context": "Lucy Ferrier had died while fleeing persecution with her father
        "question": "How did Lucy Ferrier die?",
        "answer": "fleeing persecution",
        "start_position": 28,
        "end_position": 47
    },
    {
        "context": "Sherlock Holmes used his violin playing to relax and stimulate h
        "question": "What did Holmes use to stimulate his mind?",
        "answer": "violin playing",
        "start_position": 25,
        "end_position": 39
    },
    {
        "context": "Sherlock Holmes solved the case using footprints, cigarette ash,
        "question": "What evidence did Holmes use to solve the case?",
        "answer": "footprints, cigarette ash, and cab tracks",
        "start_position": 38,
        "end_position": 79
    }
```


Fig. 9. Example *Dataset*

3) Tokenization: The next step is tokenization, which converts the QA datasets into a format suitable for processing by the BERT model during fine-tuning. A BERT tokenizer is used to generate offsets and sequence IDs from question-context pairs. Offsets provide the start and end positions of each token, while sequence IDs are vectors of 0s and 1s that indicate the relative positions of the question and context in the tokenized form. The character-based start and end positions from the QA dataset are then mapped to the tokenized context to determine the relative position of the answer. This information serves as the ground truth, as the BERT model outputs the relative positions of answers within the tokenized context during the QA task [2]. The following settings were applied for tokenization:

- Truncation is applied only to contexts.
- Maximum token length is set to 512.
- Stride length is 128.
- Padding is applied.

4) Fine-tuning: With the datasets tokenized, they were divided into training and validation sets in a 90:10 ratio. A higher proportion of data was allocated for fine-tuning compared to the standard 80:20 split because the total number of samples was limited. Additionally, 70 QA samples were reserved for inference testing later. Various fine-tuning settings were tested by adjusting the number of epochs, learning rate,

and weight decay. Ultimately, BERT was fine-tuned using the following hyperparameter settings for both the QA datasets generated manually and automatically:

- Number of Epochs: 3-10
- Learning Rate: 2×10^{-5}
- Weight Decay: 0.01

Figure 10 (*Fine-Tuning Evaluation Loss*) illustrates the training loss observed during the fine-tuning of the BERT model using the manually prepared QA dataset. Once fine-tuning was completed, the model weights and tokenizer were saved for use during inference testing.

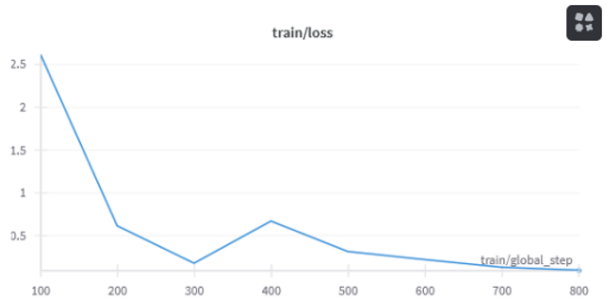

Fig. 10. Fine *tuning*

C. Evaluation

To evaluate the performance of our RAG implementations, we used BERTScore [10], as it effectively captures semantic similarity between generated and reference texts. This capability is particularly important for cases where different words express the same meaning. We began by embedding the questions from the inference dataset. Using various RAG pipelines, we generated contexts and answers based on these query embeddings. Since the inference dataset already provided ground truth for both contexts and answers, we compared the generated outputs with the reference data. Finally, we calculated BERTScore metrics, including precision, recall, and F1, and used them to compare the performance of different RAG implementations.

In addition, we applied BERTScore to evaluate the performance of the pre-trained BERT model against BERT models fine-tuned using manually and automatically generated QA datasets. For this comparison, the generated responses were directly matched against the reference answers to compute the BERTScore. To further ensure a comprehensive evaluation, we also measured performance using additional metrics, including Exact Match (EM) and F1 score.

Figure 11 (*BERT Score*) illustrates the BERT Score evaluation system and its ability in natural language processing tasks.

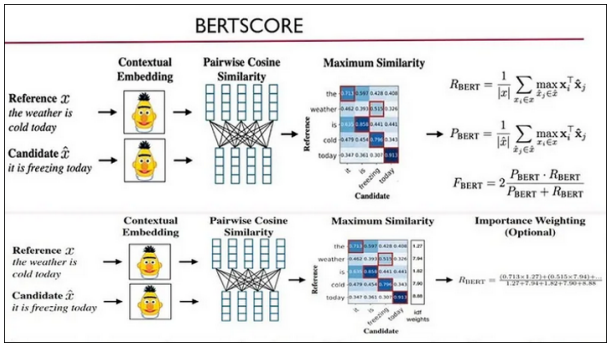

Fig. 11. BERT Score *process*

III. Results

A. Retrieval Augmented Generation

Figure 12 shows the calculated BERTScores, comparing generated contexts with reference contexts and generated answers with reference answers. The scores were consistently low across all RAG pipelines and chunking methods. The choice of chunking technique had little effect, as all methods showed similar low performance. This suggests that RAG alone is not enough to improve the performance of the pretrained bert-base-uncased for QA tasks.

The main issue seemed to be the model's limited token length size. In many cases, while RAG generated contexts that included the correct answers, the answers were often outside the model's token length, which significantly impacted its overall performance.

BERT Score Evaluation Metrics

	Recursive Chunking		Paragraph Chunking		Sentence Chunking	
	Context	Answer	Context	Answer	Context	Answer
Precision	0.3526	0.3051	0.3543	0.3166	0.3521	0.3109
Recall	0.5189	0.3537	0.5197	0.3541	0.5146	0.3276
F1	0.4193	0.3241	0.4207	0.3299	0.4175	0.3146

	Semantic Chunking		Phrase Chunking		Non-Continuous Semantic Chunking	
	Context	Answer	Context	Answer	Context	Answer
Precision	0.3882	0.3075	0.398	0.3719	0.3536	0.323
Recall	0.5012	0.3429	0.4749	0.3749	0.5038	0.3661
F1	0.437	0.3217	0.4325	0.3688	0.4149	0.3406

Fig. 12. RAG *Results*

Figure 13 provides examples of ground truth and generated texts, each containing a context, a question, and an answer. The quality of the generated texts was consistently poor across all RAG methods, including instances where semantic chunking produced completely irrelevant answers, such as a single punctuation character.

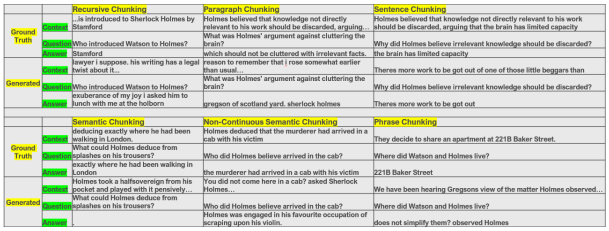

Fig. 13. RAG *Results*

Additionally, we suspect that the model itself may be too limited in capability. Testing with larger, more advanced models may demonstrate improved performance when using the same generated contexts.

B. Fine-Tuning BERT

The BERT fine-tuning process yielded interesting results and highlighted potential for future improvements. The two fine-tuned models (one trained on manually generated QA datasets and the other on automated QA datasets) were compared with the pre-trained, out-of-the-box BERT model. Both fine-tuned models showed significant improvement in their ability to answer inference questions, achieving higher BERTScores, as well as improved EM and F1 scores. These metrics are summarized in **Table I** and **Table II** below.

While these improvements are encouraging, the results were somewhat surprising. Despite the automated QA dataset containing significantly more samples, its performance improvement was only slightly better than that of the manual dataset. A possible explanation is that the higher quality of the manual dataset compensated for its smaller size. This suggests that the quality of the data provided to the model is just as important as the quantity.

Future work could focus on combining these strengths by automating dataset creation while maintaining quality. Such an approach could potentially lead to significantly better fine-tuning results.

Model Type	Precision	Recall	F1 Score
Pretrained BERT	0.4386	0.3873	0.4002
Fine-tuned BERT - QA Manual	0.5192	0.5692	0.5337
Fine-tuned BERT - QA Automated	0.5594	0.5869	0.5651

TABLE I
BERT SCORE RESULTS FOR PRETRAINED AND FINE-TUNED MODELS

Model Type	Exact Match	F1 Score
Pretrained BERT	0.0000	8.4244
Fine-tuned BERT - QA Manual	17.6471	32.5442
Fine-tuned BERT - QA Automated	19.1176	34.5161

TABLE II
STANDARD EM AND F1 SCORE COMPARISON

IV. CONCLUSION & FUTURE WORKS

In conclusion, our study successfully fine-tuned BERT for question-answering tasks using two data preparation approaches: a small, high-quality manual dataset and an automated QA dataset. Both methods improved performance, but the manual dataset's higher quality compensated for its smaller size, resulting in comparable improvements to the automated dataset. This highlights the importance of data quality alongside quantity.

However, we found that RAG alone was not sufficient for improving the performance of pretrained bert-base-uncased with enhanced context generation. The model's small token length led to exclusion of answers in contexts, limiting its effectiveness. Despite trying various chunking methods, this issue persisted.

Future work could focus on improving RAG by exploring better text embedding models for smarter chunking. Also, maintaining the quality of automatically generated datasets could significantly enhance the fine-tuning process and overall model performance.

REFERENCES

[1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805, 2018.
[2] Shawn Ding. [fine tune] fine tuning bert for question answering (qa) task, 2023.
[3] Arthur Conan Doyle. *A Study in Scarlet*. Project Gutenberg, 1887.
[4] L Elbatarny, RKG Do, N Gangai, F Ahmed, S Chhabra, and AL Simpson. Applying natural language processing to single-report prediction of metastatic disease response using the or-rads lexicon, 2023.
[5] HugginFace. Sentence-transformer, 2024.
[6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.
[7] Anurag Mishra. Five levels of chunking strategies in rag— notes from greg's video, 2024.
[8] thallyscostalat. Chunking strategies optimization for retrieval augmented generation (rag) in the context of generative ai, 2024.
[9] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.
[10] Tianyi Zhang*, Varsha Kishore*, Felix Wu*, Kilian Q. Weinberger, and Yoav Artzi. Bertscore: Evaluating text generation with bert. In *International Conference on Learning Representations*, 2020.